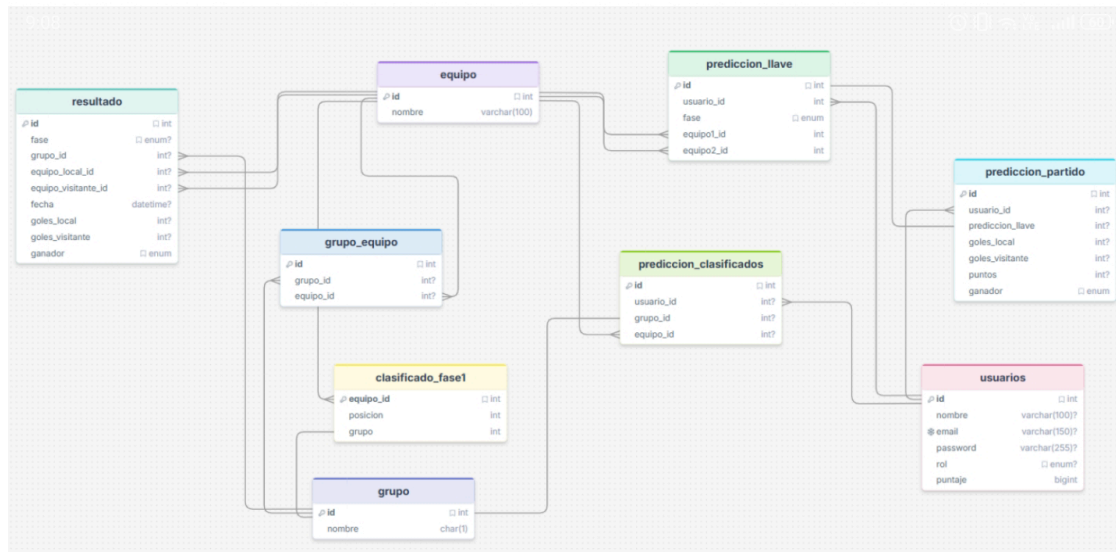


Sistema de predicciones Mundial 2026



Servicios y microservicios

Dominio: Auth & Usuario

user-service — gestiona el perfil y datos del usuario. Conecta a **postgres-user** vía JDBC/SQL. Publica **auth events** y **scoring events** hacia Kafka.

auth-service — maneja autenticación, tokens JWT y sesiones. Conecta a **postgres-auth** vía JDBC/SQL. Se comunica con Redis para gestión de tokens y publica **auth events**.

Dominio: Puntuación & Competición

league-service — administra ligas y competencias entre usuarios. Conecta a **postgres-league** y publica **league events** hacia Kafka.

ranking-service — calcula y mantiene el ranking global de usuarios. Conecta a **postgres-ranking**, consume **scoring events** y **league events** desde Kafka, y publica **ranking events**.

scoring-service — calcula puntos según aciertos. Conecta a **postgres-scoring**, consume **match events** y **league events**, y publica **scoring events** hacia Kafka.

Dominio: Torneo & Predicciones

tournament-service — gestiona la estructura del torneo: grupos, fases, equipos, llaves. Conecta a **postgres-tournament** vía JDBC/SQL. Publica **tournament events** y **match events** hacia Kafka.

prediction-service — recibe y almacena todas las predicciones del usuario. Conecta a **postgres-prediction**. Consume **match events** y publica **prediction events** hacia Kafka.

Dominio: Notificaciones

notification-service — consume múltiples eventos de Kafka (**auth events**, **match events**, **scoring events**, **ranking events**, **league events**) y dispara notificaciones a través de tres canales externos: SendGrid (email), Firebase FCM (push Android) y Apple APNs (push iOS). Persiste registros en **postgres-notification**.

Infraestructura Transversal

Kong API Gateway — punto de entrada único del sistema. Recibe todas las peticiones HTTP/REST del frontend y las enruta hacia los dominios correspondientes. Maneja autenticación de borde, rate limiting y SSL termination.

Apache Kafka — bus de eventos central. Todos los dominios publican y consumen eventos aquí: **auth**, **tournament**, **match**, **prediction**, **scoring**, **ranking** y **league events**.

Consul (Service Discovery) — registro y descubrimiento de servicios. Todos los microservicios se registran aquí para que puedan encontrarse entre sí sin hardcodear IPs.

Config Server + Vault — centraliza configuración de entornos y secretos (claves de BD, API keys, JWT secrets) para todos los servicios.

Redis Cluster — caché compartida, usada principalmente para rankings en tiempo real, standings y tokens de sesión.

Son 8 microservicios de negocio y 5 componentes de infraestructura transversal.

Primer prompt

>> estoy haciendo un sistema de ingreso de predicciones de resultados para el mundial 2026, el usuario ingresa sus predicciones para todo el mundial, incluyendo marcador final, resultado (gano, perdio, empate) de los primeros partidos y los predichos por el mismo, escoge los clasificados a 16avos sin importar los resultados anteriores pero luego de clasificaciones y para escoger las llaves se tienen en cuenta los equipos escogidos para avanzar en cada ronda, quiero que tenga en cuenta tambien los terceros mejores a la hora de ingresar la prediccion. El sistema debe tener una clasificacion de puntos y segun los aciertos una suma automatica de los mismos. Mi proyecto debe estar con una estructura en capas (controladores, servicios, repositorios y entidades), segun el modelo entidad relacion de la base de datos dada ayudame a realizar una estructura de mi proyecto con las capas necesarias, dame el codigo organizado y describe el funcionamiento de cada una de las capas y los archivos dentro de ellas.

>>

Cómo funciona cada capa

Entidades definen la estructura de datos en código. Son el espejo del modelo ER: campos, tipos, relaciones entre tablas. Nunca contienen lógica de negocio.

Repositorios son la única puerta de entrada a la base de datos. Cualquier query, búsqueda o escritura pasa por aquí. Si mañana cambias de MySQL a PostgreSQL, solo tocas esta capa.

Servicios son el corazón del sistema. Aquí vive todo lo importante: la regla de que los 8 terceros deben ser de grupos distintos, el cálculo de puntos por marcador exacto vs. solo resultado, la validación de que los equipos en una llave semifinal deben haber sido predichos en cuartos por ese mismo usuario. Los servicios llaman a repositorios, nunca a otros controladores.

Controladores son la capa más delgada. Su único trabajo es tomar lo que llegó por HTTP (req.body, req.params, req.user), pasárselo al servicio correspondiente, y devolver la respuesta JSON. Si un controlador tiene más de 15-20 líneas por método, es señal de que lógica se escapó hacia donde no debe estar.

Sistema de Predicciones Mundial de Fútbol FIFA 2026

1. Introducción

El presente documento describe la arquitectura de software del sistema de predicciones para el Mundial de Fútbol FIFA 2026. Su propósito es establecer un marco técnico de referencia que guíe al equipo de desarrollo durante el diseño, implementación, despliegue y mantenimiento del sistema. Define los componentes, sus responsabilidades, las interacciones entre ellos y las decisiones de diseño que fundamentan la solución.

El sistema permite a usuarios registrados realizar predicciones sobre los 104 partidos del Mundial FIFA 2026 (48 equipos, 12 grupos), acumular puntos automáticamente según la precisión de sus predicciones, y competir en rankings globales y ligas privadas.

2. Visión General del Sistema

2.1 Descripción del producto

El Sistema de Predicciones FIFA 2026 es una plataforma web que permite a los aficionados al fútbol registrar sus pronósticos para todos los partidos del Mundial, acumular puntuación automática y competir en tiempo real contra otros usuarios. La plataforma cubre desde la fase de grupos hasta la gran final del 19 de julio de 2026, abarcando un total de 104 partidos en 6 rondas distintas.

2.2 Objetivos del sistema

- Permitir al usuario predecir marcadores y resultados de todos los partidos del torneo.
- Bloquear predicciones automáticamente antes del inicio del mundial.
- Calcular y distribuir puntos de forma automática tras cada resultado oficial.
- Servir rankings globales y por ligas privadas con latencia inferior a 500 ms.
- Soportar hasta 100.000 usuarios concurrentes en picos de alta demanda.
- Enviar notificaciones relevantes por email y push ante eventos del torneo.

2.3 Restricciones y supuestos

- Las predicciones de todo el mundial deben registrarse antes del inicio del primer partido del torneo (11 jun 2026).
- Los resultados oficiales son ingresados exclusivamente por administradores.

3. Reglas de Negocio

3.1 Estructura del torneo FIFA 2026

El torneo se organiza en dos grandes etapas: fase de grupos y fase eliminatoria. La siguiente tabla resume la estructura completa del torneo con los nombres correctos de cada ronda:

Ronda	Nombre oficial	Partidos	Equipos	Fechas aprox.
Fase de grupos	Group Stage	72	48 → 32 clasificados	11 jun – 27 jun
Ronda de 32	Round of 32 (NUEVA)	16	32 → 16	28 jun – 3 jul
Ronda de 16	Round of 16	8	16 → 8	4 jul – 7 jul
Cuartos de final	Quarter-finals	4	8 → 4	9 jul – 10 jul
Semifinales	Semi-finals	2	4 → 2	14 jul – 15 jul
Partido 3er puesto	Third-place match	1	2 equipos	18 jul
Final	Final	1	2 → Campeón	19 jul

3.2 Reglas de clasificación

En fase de grupos clasifican los 2 primeros de cada grupo (24 equipos) más los 8 mejores terceros clasificados de entre los 12 grupos (8 equipos), sumando 32 clasificados. El bracket de la Ronda de 32 se determina automáticamente según 495 escenarios pre-definidos por FIFA (Anexo C de las regulaciones oficiales), dependiendo de qué grupos produzcan los terceros clasificados que avanzan.

3.3 Sistema de puntuación

La puntuación se calcula automáticamente al registrarse un resultado oficial. Los valores están externalizados en configuración y son modificables por el administrador.

3.3.1 Fase de grupos — por partido

Escenario	Puntos	Condición
Marcador exacto correcto	3	homeGoals90 y awayGoals90 coinciden exactamente.

Resultado simple correcto	1	Ganador o empate correcto, pero marcador incorrecto.
Predicción incorrecta	0	Ninguna de las anteriores.

3.3.2 Fase eliminatoria — por partido

Escenario	Puntos	Condición
Ganador + marcador exacto a 90 min	4	Ganador exacto + marcador a 90 min exacto, sin importar ET/penales.
Solo ganador correcto	2	Predijo el equipo ganador pero marcador incorrecto.
Predicción incorrecta	0	Equipo ganador incorrecto.

3.3.3 Predicciones especiales (bonus)

Escenario	Puntos
Acertó los 2 clasificados de un grupo (sin importar orden)	3
Acertó los 2 clasificados de un grupo EN ORDEN (1º y 2º exactos)	5
Acertó que un tercero específico avanzaría (c/u)	2
Acertó al campeón del torneo	10
Acertó al subcampeón	5
Acertó al ganador del 3er puesto	3

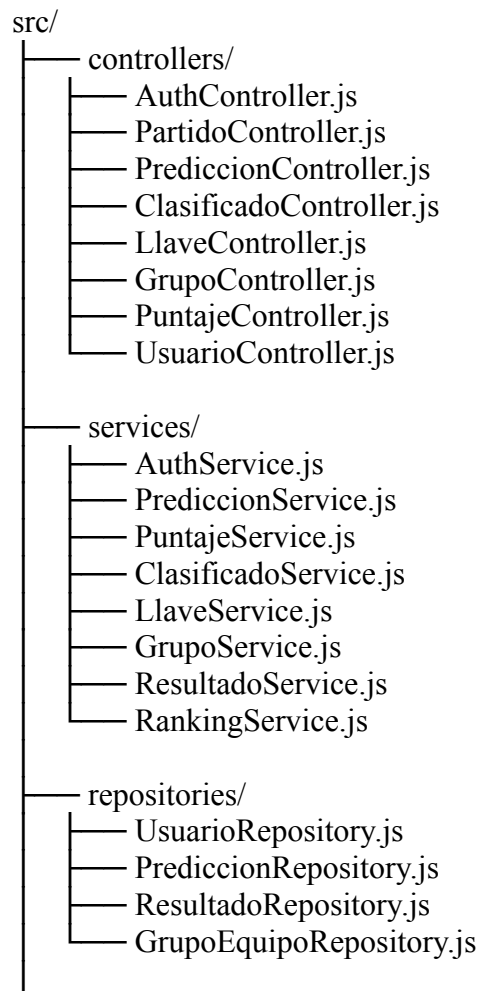
4. Requerimientos

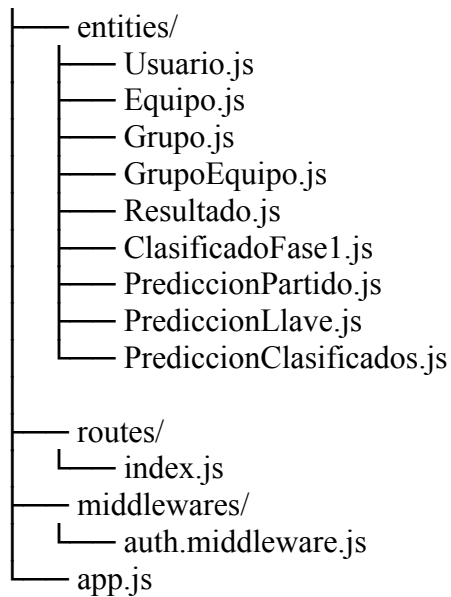
- El sistema debe gestionar: Usuarios, países, grupos, fases, predicciones, clasificaciones y puntajes.
- El usuario puede registrarse e iniciar sesión, ingresar sus datos y leer toda la información suministrada.
- Deben de visualizarse los países participantes, indicando su grupo, contrincantes, derrotas y victorias.

- Deben de visualizarse los grupos junto con su letra correspondiente y sus equipos pertenecientes.
- Las fases del campeonato deben mostrarse de manera clara, cronológica y organizada.
- Las predicciones deben ser exactas y tener los campos necesarios para crear un puntaje por usuario.
- El usuario debe poder predecir los clasificados a octavos, cuartos y semifinales.
- Debe existir un sistema de puntos adecuado para la clasificación de usuarios según sus aciertos.
- El sistema debe proporcionar información confiable y en tiempo real durante el mundial.
- Suma de puntos automática según la escala escogida luego de cada partido.
- Debe ser accesible a todo tipo de usuario interesado en participar.
- Fácil de utilizar e intuitivo a la hora del ingreso de las predicciones.

5. Arquitectura del Sistema

5.1. Estructura de carpetas





5.2 Patrones de comunicación

5.2.1 Sincrónico — REST sobre HTTP/HTTPS

Utilizado exclusivamente para la interacción cliente-servidor a través del API Gateway. El cliente envía una solicitud y espera una respuesta inmediata. Aplica para: autenticación, registro de predicciones, consulta de rankings y gestión de ligas.

6. Definición de Microservicios

5.1 auth-service

Gestiona la autenticación, el ciclo de vida de tokens JWT y el acceso con proveedores externos (OAuth2). Es la única fuente de verdad sobre identidad y credenciales.

Endpoints REST principales

Método	Path	Descripción
POST	/auth/register	Registro con email y contraseña.
POST	/auth/login	Autenticación. Devuelve access + refresh token.
POST	/auth/refresh	Renueva access token con refresh token válido.
POST	/auth/logout	Invalida el refresh token (blacklist en Redis).

POST	/auth/oauth/{provider}	Login social (Google, Apple).
GET	/auth/validate	Valida JWT (usado internamente por API Gateway).

5.2 user-service

Gestiona el perfil público del usuario, sus preferencias y sus estadísticas acumuladas de predicción. Es el bounded context de identidad de dominio (distinto de las credenciales en auth-service).

Endpoints REST principales

Método	Path	Descripción	Rol
GET	/users/me	Perfil del usuario autenticado.	USER
PUT	/users/me	Actualiza nombre, avatar, timezone, idioma.	USER
GET	/users/{userId}	Perfil público de otro usuario.	USER
GET	/users/me/stats	Estadísticas de predicción (puntos, aciertos).	USER

Entidades principales

Campo	Tipo	Descripción
id	UUID	Igual al userId del auth-service.
displayName	VARCHAR(100)	Nombre visible en rankings y ligas.
avatarUrl	TEXT	URL del avatar del usuario.
timezone	VARCHAR(50)	Zona horaria (IANA), ej: America/Bogota.
totalPoints	INTEGER	Puntos acumulados totales.
globalRank	INTEGER	Posición en el ranking global.

Eventos consumidos

Evento	Origen	Acción que ejecuta
auth.user.registered	auth-service	Crea el UserProfile inicial con los datos del registro.

scoring.user.points.updated	scoring-service	Actualiza el campo totalPoints del perfil.
-----------------------------	-----------------	--

5.3 tournament-service

Fuente de verdad sobre la estructura del torneo: equipos, grupos, partidos, standings y el bracket eliminatorio. Implementa la lógica de los 495 escenarios de bracket, el algoritmo de desempate de 7 niveles para grupos y la tabla de clasificación de terceros.

Endpoints REST principales

Método	Path	Descripción	Rol
GET	/tournament/info	Estado general del torneo.	Público
GET	/tournament/groups/{id}/standings	Clasificación de un grupo.	Público
GET	/tournament/matches	Todos los partidos (filtrable por fase/estado).	Público
GET	/tournament/bracket	Bracket completo de fase eliminatoria.	Público
GET	/tournament/third-place-standings	Ranking global de los 12 terceros.	Público
POST	/tournament/admin/matches/{id}/result	Registra resultado oficial.	ADMIN
PUT	/tournament/admin/matches/{id}/result	Corrige resultado registrado.	ADMIN

Entidades principales

Entidad	Campos clave
Match	id, phase, matchNumber, homeTeamId, awayTeamId, homeTeamSlot, awayTeamSlot, scheduledAt, status, homeGoals90, awayGoals90, wentToExtraTime, wentToPenalties, homeGoalsFinal, awayGoalsFinal, winnerId
GroupStanding	groupId, teamId, position, played, won, drawn, lost, goalsFor, goalsAgainst, goalDifference, points, fairPlayScore
BracketScenario	id, scenarioCode, thirdPlaceGroupCombination, matchMappings (JSON con los 495 escenarios), activatedAt

Eventos publicados

Evento	Topic	Descripción
match.result.registered	match.events	Resultado oficial registrado por admin.
match.result.corrected	match.events	Corrección de resultado previamente registrado.
match.started	match.events	Partido ha comenzado; bloquear predicciones.
bracket.locked	tournament.events	Bracket fijado; incluye matchMappings completos.
group standings.updated	tournament.events	Standings de un grupo actualizados.
tournament.status.changed	tournament.events	Cambio de estado del torneo (ej: GROUP_STAGE → BRACKET_LOCKING).

5.4 prediction-service

Gestiona el ciclo de vida completo de las predicciones del usuario: creación, edición, bloqueo por partido y validación de reglas por fase. No calcula puntos; esa responsabilidad pertenece al scoring-service.

Endpoints REST principales

Método	Path	Descripción	Rol
POST	/predictions	Crea o actualiza una predicción (upsert por userId + matchId).	USER
GET	/predictions/me	Todas las predicciones del usuario.	USER
GET	/predictions/me/match/{matchId}	Predicción del usuario para un partido.	USER
GET	/predictions/me/phase/{phase}	Predicciones del usuario por fase.	USER
POST	/predictions/special	Predicción especial (campeón, subcampeón, etc.).	USER

Entidades principales

Campo	Tipo	Descripción
userId	UUID	Propietario de la predicción.
matchId	UUID	Partido al que aplica.
predictedHomeGoals90	INTEGER	Goles local predichos a los 90 min.
predictedAwayGoals90	INTEGER	Goles visitante predichos a los 90 min.
predictedWinnerId	UUID	Equipo ganador predicho (obligatorio en eliminatorias).
predictedGoesToPenalties	BOOLEAN	Predicción de si habrá tanda de penales.
isLocked	BOOLEAN	true cuando el partido comenzó.

Eventos consumidos

Evento	Origen	Acción
match.started	tournament-service	Marca isLocked = true para ese matchId.
bracket.locked	tournament-service	Habilita predicciones de Ronda de 32 con equipos reales.

5.5 scoring-service

Calcula y persiste los puntos de cada predicción cuando se registra o corrige un resultado real. Garantiza idempotencia usando (matchId, userId) como clave compuesta única. Los valores de puntuación se leen desde configuración externalizada.

Entidades principales

Campo	Tipo	Descripción
userId	UUID	Usuario puntuado.
matchId	UUID	Partido al que corresponde.
predictionSnapshot	JSONB	Copia de la predicción en el momento del cálculo.
resultSnapshot	JSONB	Copia del resultado oficial.
pointsBase	INTEGER	Puntos por resultado/marcador.
pointsBonus	INTEGER	Puntos bonus (clasificadores, campeón, etc.).

pointsTotal	INTEGER	Total = pointsBase + pointsBonus.
isFromCorrection	BOOLEAN	true si el cálculo fue por corrección de resultado.

Eventos consumidos

Evento	Acción
match.result.registered	Calcula puntos para todos los usuarios que predijeron ese partido.
match.result.corrected	Anula cálculo anterior e inicia recálculo completo.
bracket.locked	Dispara cálculo de bonus por clasificadores de terceros.
tournament.status.changed (→ FINISHED)	Dispara cálculo de bonus por campeón / subcampeón / 3er puesto.

Eventos publicados

Evento	Payload resumido
scoring.user.points.updated	{ userId, matchId, pointsDelta, newTotalPoints }
scoring.match.calculated	{ matchId, phase, scoresCount, calculatedAt }
scoring.recalculated	{ matchId, affectedUsersCount, correctedAt }

5.6 ranking-service

Consolida y sirve el ranking global y por ligas privadas en tiempo real. Utiliza Redis Sorted Sets como estructura de datos principal para obtener posiciones con complejidad $O(\log N)$.

Endpoints REST principales

Método	Path	Descripción
GET	/rankings/global	Ranking global paginado con posición del usuario.
GET	/rankings/global/me	Posición y puntos del usuario autenticado.
GET	/rankings/leagues/{leagueId}	Ranking de una liga privada específica.
GET	/rankings/phases/{phase}	Ranking por fase específica del torneo.

Estrategia Redis

- Sorted Set ranking:global — score = totalPoints, member = userId.
- Sorted Set ranking:league:{leagueId} — por cada liga privada.
- Sorted Set ranking:phase:{phase} — por ronda del torneo.
- Snapshot del top 100 global cacheado cada 60 segundos.
- Actualización: ZINCRBY O(log N) al recibir scoring.user.points.updated.

5.7 league-service

Gestiona las ligas privadas entre amigos: creación, invitaciones por código y membresía. Un usuario puede pertenecer a múltiples ligas. El creador de la liga tiene rol de administrador dentro de ella.

Endpoints REST principales

Método	Path	Descripción
POST	/leagues	Crea una liga privada.
GET	/leagues/me	Ligas del usuario autenticado.
POST	/leagues/{id}/join	Unirse con código de invitación.
DELETE	/leagues/{id}/leave	Abandonar una liga.
GET	/leagues/{id}/invite-code	Obtener/regenerar código de invitación.

5.8 notification-service

Envía notificaciones transaccionales y de campaña por email y push. No contiene lógica de negocio del torneo; reacciona a eventos del sistema y delega el envío a proveedores externos.

Proveedores externos

- Email: SendGrid (o AWS SES como alternativa).
- Push Android/Web: Firebase Cloud Messaging (FCM).
- Push iOS: Apple Push Notification Service (APNs).

Eventos consumidos

Evento	Notificación enviada
auth.user.registered	Email de bienvenida con instrucciones.

match.started	Push: '¡El partido X está por comenzar! ¿Ya predijiste?'
match.result.registered	Push: 'Resultado registrado. Revisa tus puntos.'
bracket.locked	Push + Email: 'Bracket listo. Tienes hasta X para predecir la Ronda de 32.'
scoring.user.points.updated	Push: 'Ganaste N puntos. Estás en el puesto X.'
tournament.status.changed	Email masivo (inicio del mundial, final del torneo).

6. Estructura Interna MVC

6.1 Descripción del patrón por capas

Cada microservicio implementa internamente el patrón MVC extendido con las siguientes capas. La separación estricta de responsabilidades garantiza testabilidad, mantenibilidad y bajo acoplamiento.

Capa	Clase ejemplo	Responsabilidad	Tecnología
Controller	PredictionController	Recibe y valida la petición HTTP. Delega al Service. Nunca contiene lógica de negocio.	Spring MVC / @RestController
Service	PredictionService	Toda la lógica de negocio. Orquesta repositorios y publicación de eventos.	Spring @Service
Repository	PredictionRepository	Abstracción del acceso a datos. Consultas a la BD.	Spring Data JPA / @Repository
Entity	MatchPrediction	Modelo de dominio persistido. Mapeado a tabla en BD.	JPA @Entity
DTO	PredictionRequestDTO	Objeto de transferencia entre cliente y controller. Nunca exponer entidades.	Java record / POJO
Mapper	PredictionMapper	Convierte entre Entity y DTO.	MapStruct

Validator	PredictionValidator	Valida reglas de negocio desacopladas del controller.	Spring Validator / Bean Validation
EventPublisher	KafkaEventPublisher	Abstracción para publicar eventos al broker.	Spring Kafka / KafkaTemplate

7. Infraestructura y Despliegue

7.1 API Gateway — Kong

Kong actúa como el único punto de entrada al sistema. Ningún microservicio expone puertos directamente al exterior.

Función	Configuración
Validación JWT	Plugin jwt: verifica firma RS256 con clave pública de auth-service.
Rate limiting	Plugin rate-limiting: configurable por ruta (ver Sección 8.2).
Routing	Prefijos de ruta mapeados a servicios: /auth/* → auth-service:8080, /predictions/* → prediction-service:8082, etc.
CORS	Plugin cors: orígenes configurados por entorno (localhost en dev, dominio producción).
Logging	Plugin http-log: envío de access logs al stack de observabilidad (ELK).

7.2 Message Broker — Apache Kafka

Kafka opera como backbone de comunicación asíncrona entre todos los microservicios. La topología de topics y la asignación de productores y consumidores es la siguiente:

Topic	Productor	Consumidores	Particiones
auth.events	auth-service	user-service, notification-service	3
match.events	tournament-service	scoring-service, prediction-service, notification-service	12

tournament.events	tournament-service	prediction-service, notification-service, scoring-service	6
prediction.events	prediction-service	(analíticas futuras)	12
scoring.events	scoring-service	ranking-service, notification-service, user-service	12
ranking.events	ranking-service	notification-service	6
league.events	league-service	ranking-service, notification-service	3

7.3 Redis Cache — Estrategia

Qué se cachea	Estructura Redis	TTL	Invalidación
Ranking global top 100	String (JSON serializado)	60 segundos	Al actualizar puntos de cualquier usuario (refresh periódico).
Sorted Set ranking global	ZSet ranking:global	Sin TTL (actualización continua)	ZINCRBY al recibir scoring.user.points.updated.
Sorted Set por liga	ZSet ranking:league:{id}	Sin TTL	ZINCRBY por cada miembro de la liga.
Standings de grupo	Hash standings:group:{id}	30 segundos (durante partidos activos)	Invalidar al recibir group standings.updated.
Blacklist de tokens JWT	String token:{hash}	TTL = tiempo restante del token	Automática por expiración del TTL.
Rate limiting counters	String rl:{ip}:{endpoint}	Ventana configurable (ej: 15 min)	Automática por expiración del TTL.

7.4 Bases de datos por servicio

Servicio	Motor	Justificación
auth-service	PostgreSQL 16	Integridad transaccional crítica para seguridad. Soporte nativo para cifrado y auditoría.
user-service	PostgreSQL 16	Datos relacionales simples. Consistencia fuerte requerida.
tournament-service	PostgreSQL 16	Lógica relacional compleja (grupos, partidos, standings). Queries de desempate multi-nivel requieren SQL avanzado.
prediction-service	PostgreSQL 16	Volumen alto con writes predecibles. Índice único (userId, matchId) crítico.
scoring-service	PostgreSQL 16	JSONB para snapshots. Soporte de transacciones para idempotencia.
ranking-service	PostgreSQL 16 + Redis	PostgreSQL para persistencia histórica. Redis para serving en tiempo real.
league-service	PostgreSQL 16	Relaciones entre usuarios y ligas. Integridad referencial necesaria.
notification-service	PostgreSQL 16	Logs de notificaciones y preferencias de usuario.

8. Seguridad

8.1 Autenticación y autorización

El sistema utiliza JWT firmados con RS256 (clave asimétrica). La clave privada reside exclusivamente en auth-service; la clave pública es distribuida al API Gateway para verificación sin contacto con auth-service en cada petición.

Componente	Configuración
Algoritmo JWT	RS256 (clave asimétrica de 2048 bits)
Access token TTL	15 minutos
Refresh token TTL	30 días; rotación en cada uso
Claims del JWT	sub (userId), role (USER/ADMIN), email, iat, exp

Blacklist	Tokens revocados almacenados en Redis con TTL = tiempo restante
Contraseñas	bcrypt con cost factor 12 mínimo
RBAC	Claim role verificado en API Gateway; doble verificación en servicios admin

8.2 Rate limiting en API Gateway

Endpoint	Límite
POST /auth/login	5 requests / 15 min / IP
POST /auth/register	3 requests / hora / IP
POST /predictions	60 requests / min / usuario
GET /rankings/*	120 requests / min / usuario
Resto de endpoints	300 requests / min / usuario

8.3 Consideraciones OWASP Top 10

Riesgo OWASP	Mitigación aplicada
A01 Broken Access Control	RBAC con verificación en Gateway y en cada servicio. Validar que userId del token == userId del recurso solicitado.
A02 Cryptographic Failures	TLS 1.3 obligatorio. bcrypt para passwords. RS256 para JWT. Cifrado en reposo en RDS.
A03 Injection	JPA con prepared statements. Validación de inputs con Bean Validation. Sanitización en displayName y nombres de liga.
A05 Security Misconfiguration	Secretos en HashiCorp Vault. ConfigMaps en K8s sin datos sensibles. No exponer puertos de servicios internos.
A07 Identification & Auth Failures	Bloqueo tras 5 intentos fallidos. Refresh token rotation. Logout con revocación en Redis.
A09 Security Logging & Monitoring	Access logs centralizados en ELK. Alertas en Grafana para patrones anómalos (ej: brute force).

9. Riesgos y Mitigaciones

Riesgo	Probabilidad	Impacto	Mitigación
Alta concurrencia al inicio de partidos (pico de predicciones)	Alta	Alto	HPA en Kubernetes para prediction-service. Pre-escalado manual 1h antes de partidos clave.
Resultado registrado incorrectamente por admin	Media	Alto	Evento match.result.corrected + recálculo automático de puntos con scoring.recalculated.
Fallo de Kafka durante el registro de resultados	Baja	Alto	Kafka Multi-AZ (MSK). Productor con retries. DLQ para mensajes fallidos.
Inconsistencia entre scoring-service y ranking-service	Media	Medio	Consistencia eventual aceptada. Latencia máxima 10s. Endpoint /rankings/me refleja estado más reciente de PG.
Fallo de proveedor de push notifications (FCM/APNs)	Baja	Bajo	Reintentos con backoff exponencial. Notificación degradada a solo email.
Brute force en endpoint de login	Alta	Medio	Rate limiting en Kong: 5 intentos / 15 min / IP. Bloqueo temporal de IP en Redis.
Pérdida de datos de predicciones antes del mundial	Muy baja	Muy alto	Backups automáticos diarios en RDS con retención 30 días. Read replica para recuperación.

10. Referencias

- FIFA Official Competition Regulations — 2026 FIFA World Cup. Annex C: Round of 32 Bracket Scenarios (495 predefined combinations).
- Martin Fowler — Patterns of Enterprise Application Architecture (2002). Database per Service, Repository Pattern.
- Sam Newman — Building Microservices, 2nd Edition (2021). O'Reilly Media.
- Spring Boot 3 Documentation — <https://docs.spring.io/spring-boot/>
- Apache Kafka Documentation — <https://kafka.apache.org/documentation/>
- Kong Gateway Documentation — <https://docs.konghq.com/>
- Redis Documentation — Sorted Sets: <https://redis.io/docs/data-types/sorted-sets/>

- OWASP Top 10 — 2021: <https://owasp.org/Top10/>
- HashiCorp Vault Documentation — <https://developer.hashicorp.com/vault/docs>
- Kubernetes Documentation — <https://kubernetes.io/docs/>
- ESPN — 2026 FIFA World Cup format, groups, tiebreakers (2026).